TECHNICAL SPECIFICATION · V2

Regulatory Watch

Revised MVP Architecture & Milestone Plan — Client-Aligned

An AI-powered platform for autonomous monitoring, analysis, and dissemination of international regulatory changes.

Domain: Agentic AI · NLP · RegTech

Version 2.0 · Revised from v1.0 (March 2026)

Contents

1. Executive Summary & What Changed from v1 3

2. Full System Architecture 4

3. Translation Layer — Zoom-In 5

4. Current Build Status 6

5. Milestone 1 — Infrastructure & API Skeleton 7

6. Milestone 2 — Multi-Source Ingestion 8

7. Milestone 3 — Change Detection Engine 9

8. Milestone 4 — NLP & LLM Intelligence 10

9. Milestone 5 — Translation & Personalization 11

10. Milestone 6 — Dashboard & Deployment 12

11. Technology Stack Summary 13

12. Definition of Done 13

1. Executive Summary

Regulatory Watch is organized as six sequential milestones over 22 weeks. This revised architecture (**v2**) preserves the spirit of the original plan while correcting three specific mistakes that would have caused significant over-engineering or cost blow-up at scale.

What changed from v1

#	v1 position	v2 revision	Rationale
1	Kafka + Zookeeper in infrastructure	Redis broker only	JVM + partition overhead adds nothing at MVP scale. Defer until a true multi-consumer event bus is actually needed.
2	RAG semantic search + vector embeddings included in M4	Removed from MVP	RAG is a different product. Not required to detect or alert on regulatory change. Unblocks 1 week of M4 scope.
3	Single translation layer: *translate all non-English docs → then NLP*	Split into **input-side** (OPUS-MT, Fast Track only) and **output-side** (DeepL/LLM, on summaries only, lazy & cached)	LLMs read multilingual input natively — translating raw text before Deep Track is waste. Translating summaries only is ~150× cheaper than translating full documents.
4	Layer 3 (LLM Significance Score) shown but not specified	Made explicit as keystone of M3	Gates alert volume. Without it, every whitespace change fires a ChangeEvent.
5	M4 is 6 weeks (Fast Track + Deep Track + RAG + embeddings)	M4 is 4 weeks (Fast Track + Deep Track only)	With RAG removed, scope fits comfortably in 4 weeks.

Cost impact of revisions

Estimated DeepL translation cost at 200 docs/day, 30% non-English: **v1 ≈ $1,500/month** (per-character translation of high-significance raw documents). **v2 ≈ $10/month** (translation of short summaries only, cached per (event, lang) pair). *Savings: ~150×.*

2. Full System Architecture

M2 Ingestion M3 Change Detection M4 NLP / LLM M5 Personalization M6 Dashboard

Translation (split)

```
flowchart TB Sources[/"Regulatory sources: websites · PDFs · RSS · XML · email"/] subgraph M2["M2 — Ingestion"] WC["WebConnector
Crawl4AI + Playwright"] PC["PDFConnector
Docling ▸ pdfplumber"] RC["RSSConnector
feedparser"] XC["XMLConnector
lxml + USLM/Akoma"] EC["EmailConnector
imaplib + MIME"] LANG["Language detect
langdetect / fasttext"] HASH["SHA-256 hash"] PG1[("raw_documents
PostgreSQL")] S3[("Artifact store
S3 / MinIO")] end Sources --> WC & PC & RC & XC & EC WC & PC & RC & XC & EC --> LANG --> HASH HASH --> PG1 HASH -.-> S3 subgraph M3["M3 — Change Detection · 3 layers"] L1["L1 Hash check
short-circuits 90%+"] L2["L2 Semantic diff
diff-match-patch + redlines"] L3["L3 Significance Score
LLM 0.0–1.0 + change_type"] SV[("source_versions
immutable history")] CE[("change_events
diff + score")] end PG1 --> L1 L1 -- same hash --> SV L1 -- new hash --> L2 --> L3 --> CE L2 --> SV subgraph TIN["Input translation · Fast Track ONLY"] OMT["OPUS-MT
offline · free"] end subgraph M4["M4 — NLP & LLM Intelligence"] subgraph FAST["Fast Track · every document"] NER["spaCy EntityRuler
REG_ID · HS_CODE · deadlines"] CLS["BART-large-mnli
zero-shot class."] end subgraph DEEP["Deep Track · significance ≥ 0.7"] SUM["LLM summarization
reads ANY lang → English"] OBL["Obligation extraction"] KW["Keyword extraction"] end ANALYSIS[("document_analysis
summary_en · entities · category
obligations · keywords")] end CE --> TIN --> FAST CE -->|original language| DEEP FAST & DEEP --> ANALYSIS subgraph M5["M5 — Personalization & Alerts"] UPREF[("users + preferences
industries · regions · keywords
preferred_lang")] MATCH["Matching engine
keyword + semantic"] ALERTS[("alerts
per user × event")] end ANALYSIS --> MATCH UPREF --> MATCH --> ALERTS subgraph TOUT["Output translation · lazy · cached"] TR["DeepL or LLM
translates summary only
per client.preferred_lang"] TCACHE[("cache
(event_id, lang)")] end ALERTS -- lang = en --> DISP ALERTS -- lang ≠ en --> TR TR <--> TCACHE TR --> DISP DISP["Dispatch
email · Slack · webhook"] subgraph M6["M6 — Dashboard (Streamlit ▸ React)"] FEED["Feed page"] DETAIL["Detail + diff view"] PREFS_UI["Preferences UI"] ALERTS_UI["Alerts page"] FB["/feedback API"] end ALERTS --> FEED & ALERTS_UI CE --> DETAIL UPREF <--> PREFS_UI FB --> UPREF classDef m2 fill:#d7f2d7,stroke:#2a7a2a,stroke-width:2px,color:#0a2a0a classDef m3 fill:#e5d2f0,stroke:#6b2d8f,stroke-width:2px,color:#280538 classDef m4 fill:#ffe4b8,stroke:#c57700,stroke-width:2px,color:#3d2300 classDef m5 fill:#ffd4d4,stroke:#b32020,stroke-width:2px,color:#450808 classDef m6 fill:#fff4c2,stroke:#a67c00,stroke-width:2px,color:#3d2e00 classDef trans fill:#d0e5ff,stroke:#1f4e8a,stroke-width:2px,color:#08234a class WC,PC,RC,XC,EC,LANG,HASH,PG1,S3 m2 class L1,L2,L3,SV,CE m3 class NER,CLS,SUM,OBL,KW,ANALYSIS m4 class UPREF,MATCH,ALERTS,DISP m5 class FEED,DETAIL,PREFS_UI,ALERTS_UI,FB m6 class OMT,TR,TCACHE trans
```

3. Translation Layer — Zoom-In

Translation is **not a single layer**. It is two different jobs, using two different tools, at two different points in the pipeline. Understanding this split is the most important architectural decision versus v1.

```
flowchart LR RAW["Raw document
any language
(zh / fr / de / en / …)"] RAW -->|lang ≠ en| OPUS["OPUS-MT
local · free"] RAW -->|lang = en| FAST["FAST TRACK
spaCy + BART
(English-only)"] OPUS -->|English text| FAST RAW -->|ORIGINAL language| DEEP["DEEP TRACK
LLM (GPT-4o / Claude)
natively multilingual"] FAST --> ENT["entities
category"] DEEP --> SUM["summary_en
obligations"] ENT & SUM --> MERGE["document_analysis (English)"] MERGE --> ALERT["matched alert"]
ALERT -->|client lang = en| SEND["deliver"] ALERT -->|client lang ≠ en| LLM_OR_DEEPL["DeepL or LLM
translate summary only
~300 chars"] LLM_OR_DEEPL <--> CACHE[("translation cache
per event × lang")] LLM_OR_DEEPL --> SEND classDef need fill:#d0e5ff,stroke:#1f4e8a,stroke-
width:2px,color:#08234a class OPUS,LLM_OR_DEEPL,CACHE need
```

Translation job	Where	Tool	Rationale
Input-side raw doc → English (for Fast Track)	Right before spaCy / BART	OPUS-MT (local, offline, free)	Required — spaCy EntityRuler patterns and BART-mnli are English-only and cannot be replaced by multilingual variants without quality loss.
Output-side English summary → client language	Right before alert dispatch	DeepL or LLM (cached per event × lang)	Summaries are ~300 chars — translation cost is negligible, even at high volume. Cache amortizes across clients sharing a language.

Deliberately NOT translated

- **Raw documents before Deep Track.** Modern LLMs read multilingual input natively. Translating first introduces lossy paraphrase before summarization and costs ~150× more than the alternative.
- **Full raw text via DeepL.** At $25 per million characters, a single 50-page rule translation costs $2–3. Reserved exclusively for the short output summary.

4. Current Build Status

M1 INFRASTRUCTURE
100%
Complete

M2 INGESTION
95%
Email deferred

M3 CHANGE DETECTION
40%
L1 done · L2 crude · L3 missing

M4 NLP
0%
Not started

M5 PERSONALIZATION
0%
Not started

M6 DASHBOARD
0%
Not started

Progress overview

M	Milestone	Status	Progress
M1	Infrastructure & API Skeleton	Done	
M2	Multi-Source Ingestion	Done (Email deferred)	
M3	Change Detection Engine	In progress	
M4	NLP & LLM Intelligence	Planned	
M5	Translation & Personalization	Planned	
M6	Dashboard & Deployment	Planned	

Immediate next-action priority

1. **M3 Layer 3 — LLM Significance Score** (~2 h). Keystone of change detection. Without it, alert volume will drown signal.
2. **M4 Fast Track** — spaCy EntityRuler + BART zero-shot classifier (~1 day). Deterministic, cheap, runs on every document.
3. **M4 Deep Track** — LLM summary worker (~1 day). The moment alerts become human-readable.
4. **M2 gaps** — Docling for tables, S3 artifact store (~1 day combined). Unlocks reprocessability.

5. **M5 Alert matching + dispatch** (~2 days). Ships the product loop end-to-end.

Milestone 1 — Infrastructure & API Skeleton

Weeks 1–3 · 3 weeks effort · Done

Goal. A running Docker Compose stack with FastAPI, PostgreSQL, Redis, and Celery. Every subsequent component plugs into this foundation.

User stories

- **US-1.1** Start the entire platform with a single `docker compose up`.
- **US-1.2** Auto-generated API documentation at `/docs`.
- **US-1.3** Register a new regulatory domain via the REST API.
- **US-1.4** View background-task status in a monitoring dashboard.

Tasks

T1.1	Docker Compose with api · db (Postgres 16) · redis (Redis 7) · worker (Celery) · flower.	Done
T1.2	FastAPI app skeleton with lifespan handlers, CORS, structured JSON logging.	Done
T1.3	SQLModel models: Domain, Url, FetchRun, FetchAttempt, RawDocument.	Done
T1.4	Alembic migrations for reproducible schema evolution.	Done
T1.5	CRUD REST endpoints for Domain.	Done
T1.6	Celery + Redis broker, heartbeat task every 60 s.	Done
T1.7	Flower on port 5555 for Celery monitoring.	Done
T1.8	Makefile targets: up · down · migrate · logs · test.	Done
T1.9	Health endpoints: /health · /health/db · /health/redis.	Done
T1.10	README with setup and environment documentation.	Done

Test scenarios

TEST-1.1 · Full Stack Startup

Run `docker compose up`. Wait 30 seconds. All 4 services (api · db · redis · worker) remain healthy for 10 consecutive minutes.

TEST-1.2 · Domain Registration

POST /domains returns 201 with UUID. Domain persists across container restart.

TEST-1.3 · API Documentation

Swagger UI at /docs loads. Execute POST /domains from UI with valid payload returns 201.

Milestone 2 — Multi-Source Data Ingestion

Weeks 4–6 · 3 weeks effort · **Done — 95% (EmailConnector deferred)**

Goal. A working ingestion layer that pulls regulatory content from 5 source types and stores clean, structured text in the database.

User stories

- **US-2.1** Crawl regulatory websites including JavaScript-rendered portals like EUR-Lex.
- **US-2.2** PDF guidance documents extracted with tables intact.
- **US-2.3** Add RSS feeds; detect new publications within 30 minutes.
- **US-2.4** SHA-256 fingerprint on every document; no duplicates.
- **US-2.5** Respect robots.txt and apply rate limiting.

Tasks

T2.1	IngestorBase abstract class with `async def fetch() → List[RawDocument]`.	Done
T2.2	WebConnector · Crawl4AI async Playwright, Markdown output, LLM fallback, blocker-page detector.	Done
T2.3	PDFConnector · Docling (optional, opt-in via `requirements-docling.txt`) + pdfplumber fallback with per-page offsets.	Done
T2.4	RSSConnector · feedparser; Celery Beat every 30 minutes.	Done
T2.5	XMLConnector · lxml + XPath for USLM & Akoma Ntoso (schema-aware).	Done
T2.6	EmailConnector · imaplib + MIME; PDF attachments → PDFConnector.	Deferred (post-MVP)
T2.7	URL normalization: strip tracking, resolve redirects, detect spider traps.	Done
T2.8	Store RawDocument with source metadata + `artifact_uri` cold-storage pointer.	Done
T2.9	S3 artifact storage wired into `upsert_documents`; fails open when `AWS_S3_BUCKET` unset.	Done
T2.10	Connectors as Celery tasks (Web: 6h · RSS: 30m · XML/USLM: 24h; PDFs discovered transitively by web crawler).	Done
T2.11	CJK-aware language detection (`app.ingestion.lang`) — Unicode-block pre-classifier fixes the zh-Hant → ko misclassification observed in live testing.	Done
T2.12	Blocker-page detector in WebConnector — refuses CAPTCHA / Cloudflare / access-denied interstitials before they pollute `raw_documents`.	Done

Test scenarios

TEST-2.1 · Web Crawl (JavaScript Site)

Configure EUR-Lex seed. At least 10 documents ingested per cycle; extracted text ≥ 500 chars, no raw HTML tags.

TEST-2.2 · PDF Extraction with Tables

FDA guidance PDF. Table cells preserved as structured rows, not merged into a single text run.

TEST-2.3 · RSS Polling

Federal Register RSS feed. ≥ 5 new items stored after one cycle with title, date, summary, link.

TEST-2.4 · Duplicate Prevention

Ingest same URL twice. Exactly one row; second ingest updates `last_seen_at`.

TEST-2.5 · Domain Boundary Enforcement

Seed `cbp.gov/trade/`. Zero URLs outside `cbp.gov` stored.

Milestone 3 — Change Detection Engine

Weeks 7–8 · 2 weeks effort · 40%

Goal. Detect when a regulatory document has genuinely changed, compute meaningful diffs, and store version history with LLM-powered interpretation.

User stories

- **US-3.1** Notified only when regulatory content genuinely changes (no whitespace noise).
- **US-3.2** See exactly which paragraphs changed.
- **US-3.3** Plain-English explanation of what changed and required action.
- **US-3.4** Computationally efficient; monitor thousands of URLs without excessive cost.

Tasks

T3.1	**Layer 1 · Content Hashing.** SHA-256 of normalized text; skip if unchanged (eliminates 90%+ compute).	Done
T3.2	source_versions table: (source_url, content_hash, raw_text, first_seen_at, last_seen_at).	Done
T3.3	**Layer 2 · Semantic Text Diff.** diff-match-patch (char/word level) + redlines for legal-aware tokenization (`U.S.C.`, `CFR`, `§`).	Partial — stdlib `difflib` only; diff-match-patch + redlines pending.
T3.4	Serialized diff as structured JSON in change_events.unified_diff.	Done
T3.5	HTML diff renderer (green adds / red deletes) for dashboard.	Planned
T3.6	**Layer 3 · LLM Interpretation.** Structured prompt → (change_type, affected_entities, deadline_changes, compliance_summary) via Pydantic.	Planned — next priority
T3.7	significance_score field (0.0–1.0) on change_events.	Planned
T3.8	Exponential-backoff cooldown for URLs with ≥ 3 consecutive fetch errors.	Planned
T3.9	Celery Beat re-crawl schedule: daily (priority ≥ 0.7) / weekly (others).	Planned

Test scenarios

TEST-3.1 · False Positive Prevention

Ingest a document; re-ingest with only whitespace change. No new version. No alert.

TEST-3.2 · Substantive Change Detection

Modify a compliance deadline. Re-ingest. New version created, diff highlights the sentence, LLM classifies as substantive or critical.

TEST-3.3 · LLM Interpretation Output

All four fields present (change_type, affected_entities, deadline_changes, compliance_summary ≥ 50 words). Pydantic validation passes ≥ 95%.

TEST-3.4 · Version History Integrity

Three crawls with one content change → exactly 2 versions; version_number increments; diff_from_previous null for v1, non-null for v2.

Milestone 4 — NLP & LLM Intelligence

Weeks 9–12 · 4 weeks effort (revised from 6) · 0%

Goal. Extract structured intelligence from raw regulatory text — named entities, categories, keywords, summaries, and obligations — using a dual pipeline.

> **Revision from v1**
>
> RAG endpoint (T4.11) and pgvector embeddings (T4.10) have been **removed from MVP scope**. They are a separate product, not required for change detection and alerting. This reduces M4 from 6 weeks to 4 weeks.

User stories

- **US-4.1** Documents classified by topic (data privacy, financial, healthcare, trade, etc.).
- **US-4.2** Key entities extracted (regulation ID, effective date, regulatory body, penalties).
- **US-4.3** 3-paragraph executive summary for documents over 5 pages.
- **US-4.4** Each compliance obligation extracted as {actor, obligation, deadline, source_paragraph}.

Fast Track tasks (Weeks 9–10)

T4.1	spaCy 3.8 + en_core_web_trf transformer pipeline.	Planned
T4.2	Custom EntityRuler: REGULATION_ID, EFFECTIVE_DATE, REG_BODY, COMPLIANCE_DEADLINE, PENALTY_AMOUNT.	Planned
T4.3	BART-large-mnli zero-shot with candidate labels (environmental / financial / healthcare / data privacy / trade / labor / energy / cybersecurity).	Planned
T4.4	Store extracted entities as JSON in Document.entities; primary category in Document.category.	Planned
T4.5	Trigger Fast Track as Celery task per newly ingested document.	Planned

Deep Track tasks (Weeks 11–12, revised)

T4.6	Map-Reduce summarization with LangChain (chunk 2,000, overlap 200) → 3-paragraph executive summary.	Planned
T4.7	Keyword extraction via structured prompt + PydanticOutputParser (top 10 keywords).	Planned
T4.8	Obligation extraction → {actor, obligation, deadline, source_paragraph} as JSON array.	Planned
T4.9	Deep Track routing: >10 pages · Fast-Track confidence <0.7 · significance ≥ critical.	Planned

Test scenarios

TEST-4.1 · Entity Extraction Accuracy

20 real documents. ≥ 3 entities per doc; REGULATION_ID identified in ≥ 80% of documents that contain one.

TEST-4.2 · Zero-Shot Classification

50 docs with 10 human-labeled. Primary category matches human label ≥ 80%.

TEST-4.3 · Summarization Quality

50-page PDF → 150–300 words; no claim contradicts source; at least one compliance deadline present.

TEST-4.4 · Obligation Extraction

EU AI Act processed. ≥ 50 obligations extracted (benchmark: 729 with LLaMA 3.3 70B); non-empty actor and obligation fields.

Milestone 5 — Translation & Personalization

Weeks 13–16 · 4 weeks effort · 0%

Goal. Multilingual support for international regulatory documents and a personalized alert engine that matches updates to individual user interest profiles.

> **Revision from v1 — the translation split**
>
> Translation is **no longer a single layer that runs on all non-English raw documents**. It is split into an *input-side* translator (OPUS-MT, Fast Track only) and an *output-side* translator (DeepL / LLM, on short summaries only, cached per event × language). The Deep Track LLM reads multilingual input natively and does not require translation.

User stories

- **US-5.1** Documents from non-English regulatory bodies usable by English-only NLP components.
- **US-5.2** Pharmaceutical analyst receives only pharmaceutical-related alerts.
- **US-5.3** Configure monitoring profile by industries, jurisdictions, regulatory bodies, keywords.
- **US-5.4** Alerts match conceptually-related regulations, not just exact keywords.
- **US-5.5** Choose notification frequency (immediate / daily digest / weekly digest).

Translation tasks (Weeks 13–14, revised)

T5.1	TranslationService · Input-side OPUS-MT (Helsinki-NLP, offline, 1,000+ pairs).	Planned
T5.2	Language detection on every ingested doc; store in Document.language.	Done
T5.3	If `language ≠ en`: OPUS-MT → Fast Track input. Deep Track LLM reads original.	Planned
T5.4	Regulatory glossary (≥ 50 terms) used as OPUS-MT preprocessing hints and LLM prompt context.	Planned
T5.5	**Output-side**: DeepL or LLM translates `summary_en` only, per `client.preferred_lang`. Cache table: `change_event_translations (event_id, lang, text)`.	Planned

User matching tasks (Weeks 15–16)

T5.6	User model: email · preferences (industries, bodies, keywords, jurisdictions) · notification_frequency (immediate / daily / weekly) · preferred_lang.	Planned
T5.7	Alert model: user_id · document_id · match_score · match_reason · sent_at · read_at.	Planned
T5.8	Keyword matching: SQL predicates over document entities and category.	Planned
T5.9	Semantic matching: embed interest description with `BAAI/bge-small-en-v1.5`; pgvector cosine similarity > 0.75.	Planned

T5.10	Notification Celery task; batch by user + frequency; send via SMTP / SendGrid.	Planned

Test scenarios

TEST-5.1 · Translation Quality

BaFin circular (de) and AMF notice (fr). Language correctly detected. Translated text is coherent English; regulatory glossary terms preserved; original text still available.

TEST-5.2 · Keyword Matching Precision

Profile with {pharmaceutical, FDA, drug approval}. 20 mixed docs → 10 pharmaceutical alerts, 0 false positives.

TEST-5.3 · Semantic Matching Recall

Profile "carbon emissions and greenhouse gas reporting". Doc about "CO2 emission thresholds" (no "greenhouse gas" keyword) → alert with match_reason=semantic, score ≥ 0.75.

TEST-5.4 · Notification Frequency

Two users (immediate / daily). Five matching docs. After 1h: immediate user has 5 emails; daily user has 0 (pending digest).

Milestone 6 — Dashboard & Production Deployment

Weeks 17–20 · 4 weeks effort · 0%

Goal. A usable 5-view dashboard, full end-to-end integration tests, and production deployment on a VPS with CI/CD.

User stories

- **US-6.1** Browse chronological feed of regulatory updates with category badges and AI summaries.
- **US-6.2** Search across all monitored documents with typo tolerance.
- **US-6.3** Side-by-side color-coded diff when clicking an updated document.
- **US-6.4** Configure monitoring preferences through a web interface.
- **US-6.5** Mark alerts as read and filter by jurisdiction or category.

Dashboard tasks

T6.1	Streamlit multi-page app + Meilisearch client + FastAPI client.	Planned
T6.2	Feed Page · chronological cards · category badges · AI summary · filters.	Planned
T6.3	Detail Page · summary · entities · keywords · obligations · side-by-side diff.	Planned
T6.4	Search Page · Meilisearch sub-50ms · facets · snippet highlights.	Planned
T6.5	Preferences Page · industries, bodies, keywords, jurisdictions, frequency.	Planned
T6.6	Alerts Page · sorted by match_score · mark-as-read · unread count.	Planned
T6.7	Plotly charts: documents per day, category distribution.	Planned

Integration & deployment tasks

T6.8	End-to-end integration tests (pytest) covering full pipeline.	Planned
T6.9	GitHub Actions CI/CD · ruff · mypy · integration tests on every PR.	Planned
T6.10	Deploy to VPS (Hetzner CX21) · Nginx reverse proxy · HTTPS via Let's Encrypt.	Planned
T6.11	Structured JSON logging shipped to aggregator or rotating files.	Planned
T6.12	Architecture diagram + final README + deployment instructions.	Planned

Test scenarios

TEST-6.1 · End-to-End Pipeline

Published EU regulation → crawled → change-detected → NLP classified "data privacy" → summarized → translation-checked → matched 2 users → alerts appear in dashboard → users view detail page. All 9 steps within 10 minutes.

TEST-6.2 · Search Relevance

100+ indexed docs. Top-3 results genuinely relevant for {GDPR, FDA, Basel} queries. Latency < 100 ms.

TEST-6.3 · Diff Visualization

Detail Page renders side-by-side diff: green additions, red deletions. LLM compliance summary below.

TEST-6.4 · CI/CD Pipeline

PR adding a connector runs lint + type-check + integration tests. Pipeline ≤ 10 minutes. Merge blocked on failure.

11. Technology Stack Summary

Layer	Technology	Justification
Backend	FastAPI + SQLModel	Async, typed, auto-docs. SQLModel by FastAPI's author.
Database	PostgreSQL 16 + pgvector	Structured + vector data in one DB. Single operational surface.
Task Queue	Celery + Redis 7	Async crawl scheduling, notification delivery. *No Kafka at MVP.*
Web crawling	Crawl4AI + Playwright	JS rendering; 13 KB avg clean markdown per page in CBP benchmarks.
PDF	Docling + pdfplumber	97.9% table accuracy (Docling); pdfplumber fallback for edge cases.
Change detection	diff-match-patch + redlines	Google-scale diffing plus legal-aware tokenization (`U.S.C.`, `CFR`, `§`).
NLP Fast Track	spaCy 3.8 + EntityRuler + BART-large-mnli	Deterministic, cheap, per-document tagging.
NLP Deep Track	LangChain + GPT-4o / Claude Sonnet	Multilingual-native summarization; structured obligation extraction.
Embeddings	BAAI/bge-small-en-v1.5	Strong retrieval, 384-dim, small footprint. Used for semantic matching only.
Translation (input)	OPUS-MT (Helsinki-NLP)	Offline, free, 1,000+ language pairs. Fast Track input only.
Translation (output)	DeepL API or LLM	Per-client summary translation; cached per event × lang; ~150× cheaper than full-doc approach.
Search	Meilisearch	Sub-50ms typo-tolerant faceted search.
Frontend	Streamlit → React	Zero-JS MVP; production flexibility later.
Infrastructure	Docker Compose + GitHub Actions	One-command deployment + CI/CD.

12. Definition of Done

A milestone is considered done when all of the following are met:

1. All tasks in the milestone have a merged pull request on `main`.
2. All test scenarios for the milestone pass without manual workarounds.
3. `docker compose up` produces a fully functional stack without errors.
4. Code coverage for new modules is ≥ 70%.
5. Documentation (README, docstrings, environment variables) is updated.
6. No high-severity security vulnerabilities in dependencies (`pip audit`).

Closing note

This v2 revision preserves the disciplined, test-driven spirit of the original plan while eliminating two sources of over-engineering (Kafka, RAG-in-MVP) and correcting the single biggest cost risk (full-document translation). The resulting architecture is simpler, cheaper at scale, and remains professionally sound for the regulatory-watch problem domain.

Regulatory Watch · Revised Architecture v2.0 · Ready for Development